

Algoritmos e Programação de Computadores I

Estruturas e Enumerações em C

Prof. Marcelo Farias

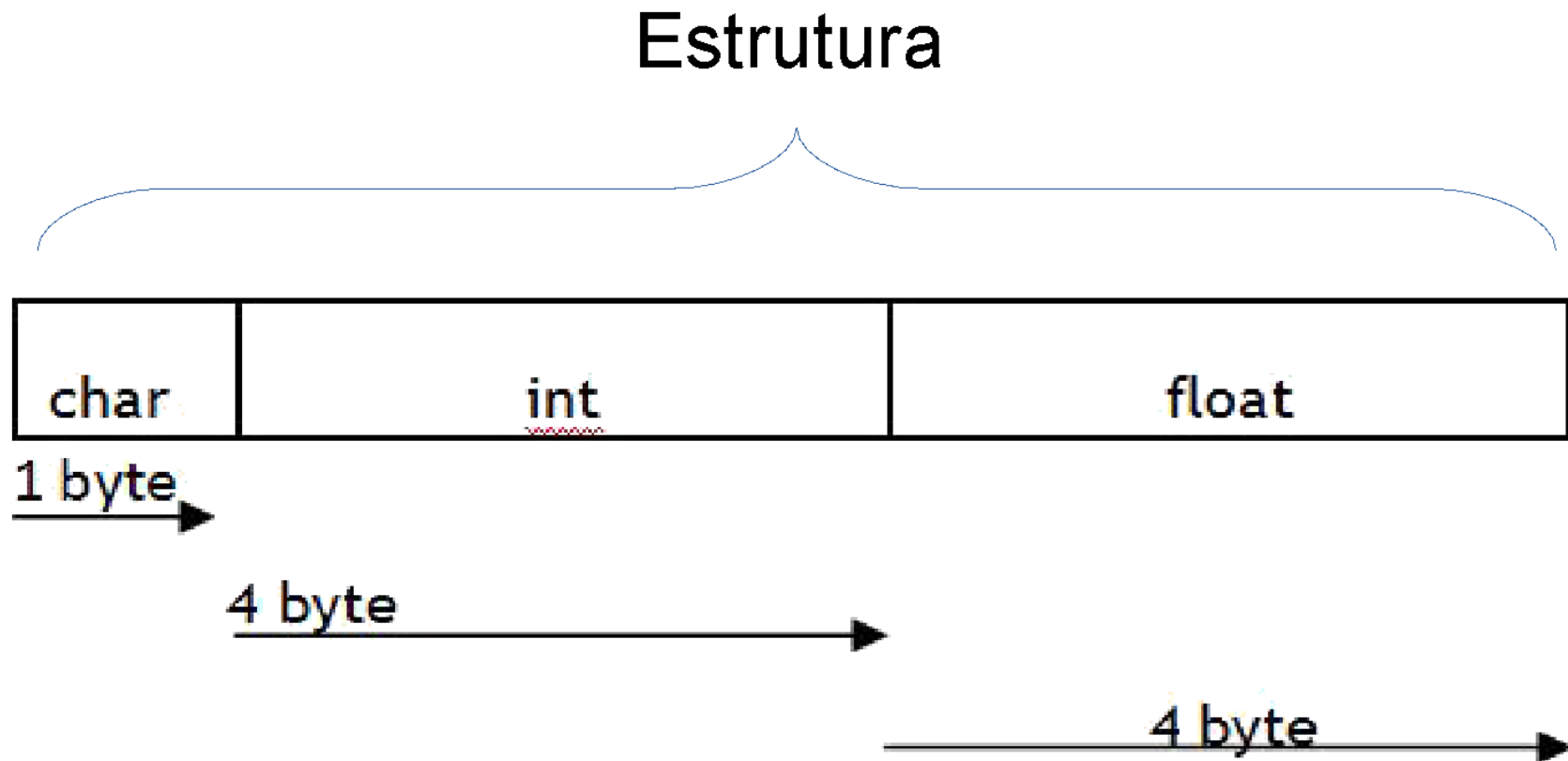
Objetivos da Aula

- Compreender o conceito de estruturas e enumerações em C.
- Aprender a declarar, inicializar e acessar os campos de uma estrutura.
- Aprender a definir conjuntos nomeados de constantes inteiras.

Estrutura (ou *Struct*)

- Um agrupamento de valores de diversos tipos.
- Cada valor é referenciado por um campo definido dentro da estrutura.
- Em geral, todos os campos da estrutura estão relacionados logicamente.
- Permite definir tipos de dados personalizados (ex.: contato, endereço, etc.).

Alocando uma Estrutura na Memória



Definindo uma Estrutura em C

```
struct <identificador> {  
    <tipo> <nome_da_variável>;  
    <tipo> <nome_da_variável>;  
    <tipo> <nome_da_variável>;  
    ...  
} <variáveis_estrutura>;
```

Obs.: ou <identificador> ou <variáveis_estrutura> podem ser omitidos.

Exemplo de Estruturas

// declaração completa

```
struct contato_t {  
    char nome[31];  
    int telefone;  
} contato;
```

// declaração sem identificador

```
struct {  
    char nome[31];  
    int telefone;  
} contato;
```

// declaração sem variável

```
struct contato_t {  
    char nome[31];  
    int telefone;  
};
```

// declaração só da variável

```
struct contato_t contato;
```

Acessando Campos da Estrutura

- Operador . permite acessar um campo da estrutura.
- Forma geral:
<variável_estrutura>.<nome_da_variável>
- Permite manipular um campo individualmente.

Exemplo de Acesso aos Campos

```
struct endereco_t {  
    char logradouro[31];  
    int numero;  
    unsigned long int cep;  
    char bairro[16];  
} endereco;
```

```
strcpy(endereco.logradouro, "SGAS");  
endereco.numero = 613;  
endereco.cep = 70200730;  
strcpy(endereco.bairro, "Asa sul");
```


Atribuição de Estruturas

- Operador = permite que os valores contidos em uma variável de estrutura sejam atribuídos a outra.
- Evita a necessidade de atribuir valores a cada campo separadamente.

Exemplo de Atribuição de Estruturas

```
struct contato_t {  
    char nome[31];  
    unsigned long int telefone;  
} contato1, contato2;
```

```
strcpy(contato1.nome, "JOSE");  
contato1.telefone = 33332222;
```

```
contato2 = contato1;  
printf("contato1: %s %lu\n", contato1.nome, contato1.telefone);  
printf("contato2: %s %lu\n", contato2.nome, contato2.telefone);
```

Estruturas Aninhadas

- Uma estrutura referenciada dentro de outra estrutura.
- Uso do operador . em sequência para acessar os campos em cada nível de aninhamento.

Exemplo de Estruturas Aninhadas

```
struct endereco_t {  
    char logradouro[31];  
    int numero;  
    unsigned long int cep;  
    char bairro[16];  
};  
  
struct contato_completo_t {  
    char nome[31];  
    unsigned long int telefone;  
    struct endereco_t endereco;  
}  
  
contato;  
contato.endereco.numero = 613;
```

Matrizes de Estruturas

- Cada elemento da matriz armazena um conjunto de informações diferentes.
- Tipicamente usado para armazenar listas de dados na memória.

Exemplo de Matriz de Estruturas

```
#define MAX 10
```

```
struct endereco_t {  
    char logradouro[31];  
    int numero;  
    unsigned long int cep;  
    char bairro[16];  
} endereco;
```

```
struct endereco_t enderecos[MAX];
```

```
for(int i=0; i < MAX; i++)  
    memset(&enderecos[i], 0, sizeof(struct endereco_t));
```

Enumeração (ou *Enum*)

- Um conjunto de constantes inteiras que especifica os valores permitidos por uma variável.
- Cada constante inteira é associada a um nome.
- Valor da primeira constante é 0, sendo depois incrementado em 1 na próxima constante.
- Pode-se atribuir um valor específico a qualquer uma das constantes.

Definindo uma Enumeração em C

```
enum <identificador> {  
    <nome_da_constante>=<valor>,  
    <nome_da_constante>=<valor>,  
    <nome_da_constante>=<valor>,  
    ...  
} <variáveis_enumeração>;
```

Obs.: ou <identificador> ou <variáveis_enumeração> pode ser omitido. Não é permitido declarar a mesma constante em outra enumeração.

Exemplo de Enumerações

/ declaração completa com valor iniciando em 0 */*

enum status {ATIVO, INATIVO} situacao_aluno;

/ declaração completa com valores */*

enum status {ATIVO=1, INATIVO=0} situacao_aluno;

/ declaração sem identificador */*

enum {ATIVO, INATIVO} situacao_aluno;

/ declaração sem variável */*

enum status {ATIVO, INATIVO};

/ declaração só da variável */*

enum status situacao_aluno;

Como usar Enumerações?

- Definir um conjunto finito de valores (ex.: dias da semana, meses, etc.).
- Indexar elementos de uma matriz.
- Validar entrada de dados.

Exemplo de Uso de Enumerações

```
char nomes[7][8] = {"domingo", "segunda", "terça", "quarta", "quinta",  
"sexta", "sabado"};
```

```
enum sexo {MASCULINO = 1, FEMININO = 2};  
enum dias_da_semana {DOM, SEG, TER, QUA, QUI, SEX, SAB} dia;
```

```
scanf("%u", &dia);  
if(dia >= DOM && dia <= SAB) printf ("hoje é %s\n", nomes[dia]);  
else printf ("dia inválido\n");
```

Enumeração bool

- Tipo introduzido no padrão C99 com a biblioteca `<stdbool.h>`.
- Pode assumir os valores `false` (0) e `true` (1).

typedef enum {false, true} bool;

Mapa da aula

Aula (teoria): Aula15–16 — Struct e enum

Laboratório guiado: Pratica10

Atividade avaliativa: Atividade04 / Trabalho Final

Sugestão de ritmo:

1) Conceitos nos slides 2) Prática no lab 3) Atividade sem 'receita'

Para estudar no livro

Livro-base: ANTONELLO, Ricardo. Linguagem C. 1. ed.

Leitura indicada: Cap. 8 (organização de dados) + Schildt (struct/enum).
O Antonello não aprofunda struct/enum: use estes slides + Prática10.

Exercícios do livro: Aplique no Trabalho Final.

Revise o capítulo após a aula e antes da prática.

Desafio (8 min)

Modele struct Produto { codigo, nome, preco } e enum Disponibilidade. Cadastre 3 produtos e liste.

Trabalhe em dupla (instrução por pares).

Ao final, uma dupla compartilha a solução com a turma.

Quiz rápido (3 perguntas)

- 1) Para que serve o operador '.' na struct?
- 2) Enum melhora o quê frente a 'números mágicos'?
- 3) Cite um exemplo de vetor de struct.

Respostas no quadro / Mentimeter / Kahoot.

Referências

- SCHILDT, Herbert. C: Completo e Total. 3. ed. Pearson Makron Books, 1997.
- ANTONELLO, Ricardo. Linguagem C. 1. ed. Livro Digital, 2020.